

Izmir Institute of Technology

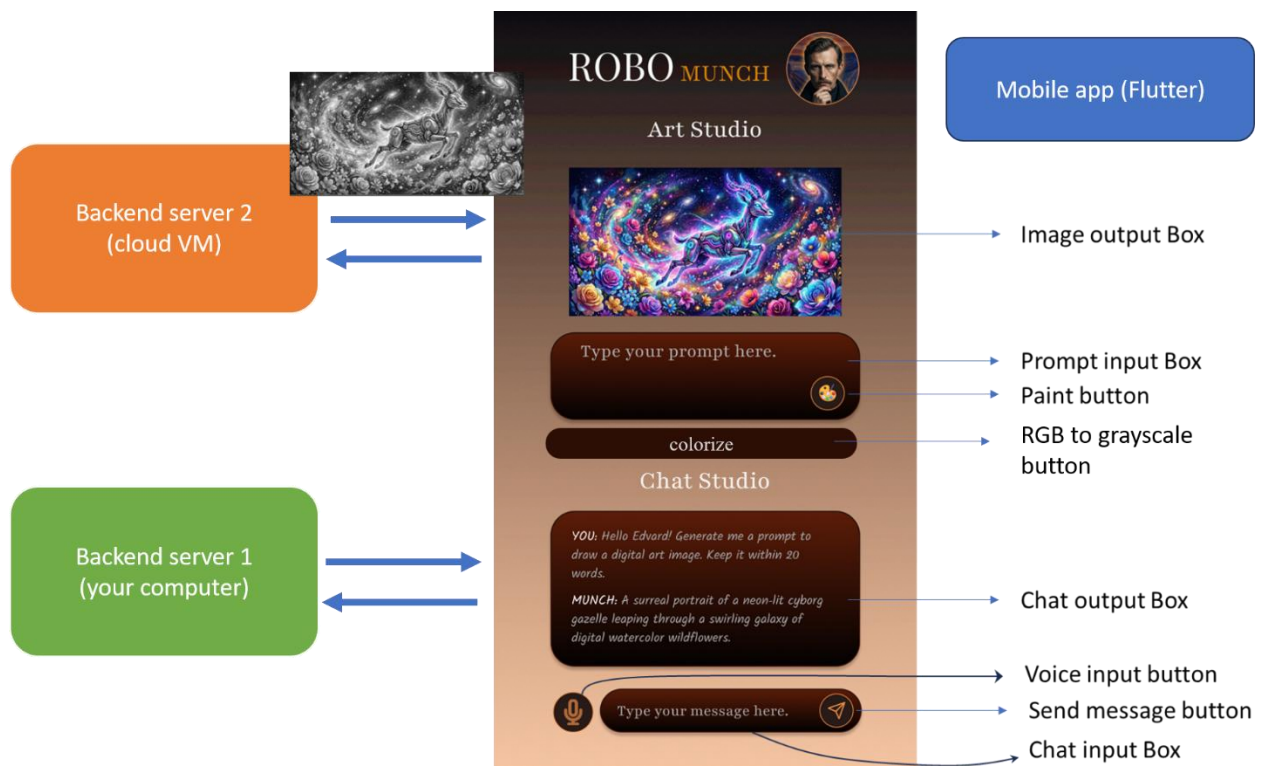
Electrical and Electronics Eng. Dep.

EE 471 Modern Software Development Practices and Technologies

Mini Project #4

Deadline: Sunday 07/06/2026 @23:59

(Fullstack Soln) - RoboMunch: An artist Chatbot Expanded



You are responsible for modifying/developing the simple artist chatbot, RoboMunch, you developed in Project #3 as a full-stack solution, and improving it with a cloud-based backend solution with a working CI/CD pipeline.

Task 1 (Flutter mobile app + localhost Backend): Develop the frontend structure you built in Project #3 and move to Flutter run on a mobile app. User will press "Voice input button" to record a message. The recorded message will be converted to text in Flutter and show up on "Chat input box". After "send message button" is pressed this text message will be sent to *Backend server 1 that will be running on your local computer*. **VERY IMPORTANT:** Both your mobile phone and computer should be connected to the same network (and requests should be sent to the computer's local IP address starting with 192.....) Your localhost backend server that is running your favorite Text Generation Inference (TGI; SmolLM2-360M-Instruct is recommended due to its lightweight) will then create a reply and send it back to the user (Flutter app). Both responses will show up on "Chat output box". When the user writes a prompt in the "Prompt input box" and press "Paint button" the prompt will be sent to *Backend server 1 (at localhost)* to create an image via a text to image generation tool like Stable Diffusion or Flux (if your HW GPU-equipped). This generated image will be sent back to the mobile app and appear in the "Image output box".

Task 2 (Flutter mobile app + Cloud Backend): Add a new "colorize" button below "Prompt input box". That button will send a request to *Backend server 2 (in the cloud)* with your AI-generated image to ask the server to either get information about the image resolution or to convert the image to grayscale, and receive it back in the mobile app and show the converted image to user.

To this end, create a Virtual Machine (VM) in your most favorite cloud service (e.g. Azure). Create a git repository with the name backend-cloud and construct a CI/CD structure for this repo similar to what we did at In-class exercise Week#12 (CI workflows for semantic versioning and Lint testing & CD workflow for deployment of backend container to the VM in the cloud).

Set up this Linux-based VM on the cloud to serve a Django-based backend server with 2 URLs:

1) One for returning the received image's resolution:

<http://YOUR-VM-IP:ASSIGNED-PORT/get/resolution>

After you ensure this endpoint works well, run the CI workflows to version it as X.Y.Z with a Lint test and then run the CD workflow to deploy this container in the cloud.

2) One for converting the sent image to grayscale and returning it to the Flutter app.

<http://YOUR-VM-IP:ASSIGNED-PORT/convert/grayscale>

After you ensure this endpoint works well, run the CI workflows to version it as X.Y+1.Z with a Lint test and then run the CD workflow to deploy this container in the cloud.

Hint: Eduroam and University Wifi access may be restricted, use hotspot (or your wifi outside for example at home) instead if you encounter with any connection issue.

VERY IMPORTANT: Use standard, low-tier Linux images (on the cloud) and do not forget to shut down or delete the VM resources once you completed this assignment.

Deliverables:

- 1) The link for your public github repositories *Backend server 1, Backend server 2, Flutter app*
- 2) A screen recording video you explaining the code blocks in your repositories.
- 3) A screen recording video you run the mobile app with its full functions: Speech to text translation, prompt generation, text to image generation, receiving the resolution of a sent image, and converting the sent image to grayscale with the colorize button.
- 4) A screen recording video showing you demonstrating the Docker logs of your *backend server 2* (the one in the cloud) when a request is sent from the mobile app.